Polymorphism

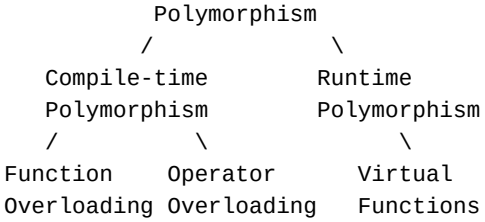
What is Polymorphism?

Polymorphism comes from the Greek words *poly* (many) and *morphism* (forms). It is the ability of an entity to take **more than one form**.

In C++, polymorphism means that a single function name, operator, or interface can behave differently depending on the context — specifically, on the type of data or object it is working with.

Types of Polymorphism

Polymorphism in C++ is divided into two main categories:



1. Compile-Time Polymorphism (Static Binding / Early Binding)

The function to be called is determined at **compile time** based on the number and types of arguments. This is also called **static binding** or **early binding**.

Function Overloading

The same function name is used with different parameter lists:

```
class Calculator {
public:
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
};
```

The compiler decides which version of `add` to call based on the arguments passed.

Operator Overloading

Giving an existing operator a new meaning for a user-defined type:

```
class Complex {
    float real, img;
public:
    Complex operator+(Complex c) {
        Complex t;
```

```
        t.real = real + c.real;
        t.img = img + c.img;
        return t;
    }
};

Complex c1(2, 3), c2(1, 4);
Complex c3 = c1 + c2;    // calls the overloaded + operator
```

The + operator now works on Complex objects just as it does on integers.

2. Runtime Polymorphism (Dynamic Binding / Late Binding)

The function to be called is determined at **runtime** based on the actual type of the object. This is achieved using **virtual functions** and is called **dynamic binding** or **late binding**.

Virtual Functions

A **virtual function** is a member function in the base class that can be overridden in a derived class. It is declared using the `virtual` keyword.

```
class Shape {
public:
    virtual void draw() {
        cout << "Drawing a shape";
    }
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle";
    }
};

class Rectangle : public Shape {
public:
    void draw() {
        cout << "Drawing a rectangle";
    }
};

int main() {
    Shape* ptr;
    Circle c;
    Rectangle r;

    ptr = &c;
    ptr->draw();    // calls Circle's draw() — runtime decision

    ptr = &r;
    ptr->draw();    // calls Rectangle's draw() — runtime decision
}
```

Without `virtual`, both calls would have called `Shape::draw()`. With `virtual`, C++ determines at runtime which version to call based on the actual object being pointed to.

Rules for Virtual Functions

- Virtual functions must be members of a class
- They cannot be static members
- They are accessed using object pointers
- A virtual function in the base class should be defined even if it is not used there
- The function signature (name and parameters) must be identical in the base and derived class
- Constructors cannot be virtual, but destructors can be

Compile-Time vs Runtime Polymorphism

Feature	Compile-Time	Runtime
Binding	Static (early)	Dynamic (late)
Mechanism	Function/operator overloading	Virtual functions
Performance	Faster (resolved at compile time)	Slightly slower (resolved at runtime)
Flexibility	Less flexible	More flexible
When to use	When types are known at compile time	When behaviour depends on object type at runtime